DeliverOpt India

Intelligent Delivery Route & Order Optimization Platform

PROJECT DOCUMENTATION · DESIGN & ANALYSIS OF ALGORITHMS (DAA)

Member	Name	Role	Codename
Member 1	**Piyush Mahapatra**	Core Data & Order Management	The Foundation Layer
Member 2	**Varun Verma**	Graph & Routing Engine	The Brain
Member 3	**Agamya Jain**	Optimization, Comparison & Visualization	The Analyst

Project folder
t3-daa-project

Document version
1.0

Date
4 November 2026

Stack
C++17 · Python/Flask · JavaScript

Table of Contents

1. Executive Summary . . . 3
2. Problem Statement & What the Project Solves . . . 3
2.1 The problem . . . 3
2.2 Problems solved and how . . . 4
3. Target Audience . . . 4
4. What the System Does - Feature Overview . . . 5
5. System Architecture . . . 6
5.1 Request life-cycle . . . 6
5.2 Design decisions worth noting . . . 6
6. Technology Stack . . . 7
7. Project Structure . . . 8
7.1 File-to-role mapping . . . 8
8. How It Works - Module Walkthrough . . . 9
8.1 Building the road network (graph model) . . . 9
8.2 Shortest path - Dijkstra's algorithm . . . 9
8.3 Multi-stop delivery - the TSP planners . . . 10
8.4 Orders, priority and driver assignment . . . 10
8.5 Load planning - 0/1 Knapsack (dynamic programming) . . . 11
8.6 Benchmarking and comparison . . . 11
9. Data Structures & Algorithms Used . . . 12
9.1 Algorithms catalogue . . . 12
9.2 Data structures catalogue . . . 13
10. Team Roles & Responsibilities . . . 14
11. User Flow . . . 17
11.1 Step-by-step by tab . . . 17
11.2 Example session - a dispatcher's morning . . . 18
12. REST API Reference . . . 18
13. Verified Sample Results . . . 19
14. Installation & Running . . . 21
14.1 Quick start - Python only (no compiler) . . . 21
14.2 Full setup - C++ engine (Windows) . . . 21
15. Known Limitations & Future Scope . . . 21
15.1 Current limitations . . . 21
15.2 Future enhancements . . . 21
16. Conclusion . . . 22
Appendix - Glossary . . . 22

1. Executive Summary

DeliverOpt India (also named *Delivery Optimizer* in the repository) is a web-based logistics decision-support tool that answers four everyday questions of a delivery business: **which order should go first**, **which driver should carry it**, **which route should the vehicle take**, and **what should be loaded into the vehicle**. It models a road network of up to 40 real Indian cities, and solves these questions with classical Design & Analysis of Algorithms (DAA) techniques: priority scheduling, greedy matching, Dijkstra's shortest path, travelling-salesman heuristics (nearest neighbour and 2-Opt), exact dynamic programming (Held-Karp), and 0/1 knapsack.

The project has a **three-layer architecture**: a browser dashboard (HTML/CSS/JavaScript with Leaflet maps and Chart.js charts), a lightweight Flask REST server, and an algorithm engine that exists in **two interchangeable forms** - a compiled C++17 binary for speed, and a pure-Python port that is used automatically when the binary has not been built. The user never needs to choose; the server detects the engine at start-up.

At a glance

Item	Detail
Domain	Last-mile / multi-stop delivery and fleet logistics
Core algorithms	Dijkstra · Nearest Neighbour · 2-Opt · Held-Karp DP · 0/1 Knapsack DP · Greedy assignment · Priority ordering · Union-Find
Key data structures	Adjacency list · Min-heap · Max-heap priority queue · Hash maps · DP tables · Bitmasks · Disjoint-set
Interface	Single-page dashboard with 4 tabs: Route Planner, Orders & Drivers, Load Planner, Benchmarks
Back end	Flask (Python) exposing 6 POST JSON endpoints plus a mode check; C++17 engine via subprocess, Python fallback
Size	About 2,400 lines of source: Python ≈ 500, C++ ≈ 950, HTML/CSS/JS ≈ 950
Team	3 members, each owning one layer: Foundation (data), Brain (routing), Analyst (optimization & analysis)

2. Problem Statement & What the Project Solves

2.1 The problem

A dispatcher at a courier or distribution company faces a stream of orders with different urgencies, a limited set of drivers with limited vehicle capacity, and a road network where the order in which stops are visited changes the total kilometres (and therefore fuel cost and delivery time) dramatically. Doing this by hand or by gut-feel produces three costly outcomes: urgent parcels are delayed, vehicles are loaded badly or run empty, and drivers take unnecessarily long routes.

Mathematically, the hard part is the **Travelling Salesman Problem (TSP)**: visiting m stops and returning to the depot has m!/2 distinct tours (m stops plus the depot), which is NP-hard. Even 12 stops already means hundreds of millions of orderings, so naïve trial of every route does not scale. The project demonstrates how algorithm design (heuristics versus exact dynamic programming) handles this trade-off between **speed and optimality**.

2.2 Problems solved and how

Real-world problem	How the app solves it	Technique
Urgent parcels must not wait behind routine ones	Orders carry a priority (Urgent / Normal / Low) and are served strictly in that order	Priority ordering (heap / sort)
Which driver takes which parcel without overloading the vehicle?	Each order, highest priority first, goes to the first driver whose free capacity is sufficient; otherwise marked 'No capacity'	Greedy first-fit
What is the fastest / shortest way from city A to city B?	Shortest path over weighted road graph, drawn on the map with distance, drive time and fuel cost	Dijkstra with min-heap
In what order should one vehicle visit several stops and return?	Three selectable planners: Quick (nearest neighbour), Better (2-Opt), Best (exact, ≤ 12 stops)	NN, 2-Opt, Held-Karp DP
Which parcels should be packed when not everything fits?	Chooses the subset with maximum total value within the vehicle's weight limit	0/1 Knapsack DP
Which algorithm should I trust, and does it scale?	Benchmark tab runs all three planners on networks of chosen sizes and plots cost and run time	Benchmarking, comparison

3. Target Audience

Audience	Why they need it	What they get
Small & mid-size courier / logistics operators	Plan daily runs without expensive routing software	Quick, explainable routes with km, time and fuel estimates; fair driver loading
Dispatchers & fleet supervisors	Decide priorities, driver assignment and load packing under time pressure	One dashboard for orders, drivers, routes and load planning
E-commerce / D2C sellers with own fleet	Ship parcels from a hub to multiple cities economically	Multi-stop tour planner and knapsack load planner
DAA students & faculty	See textbook algorithms (Dijkstra, TSP, DP, greedy) working on a realistic problem	Side-by-side comparison of heuristic and exact methods with live charts
Developers & researchers	A clean reference of a C++ engine behind a JSON/Flask API with a Python twin	Readable, modular code and a clear command interface

> **Scope note.** The project is a prototype / teaching platform. It uses straight-line (great-circle) distances scaled to road distance, not live traffic or live map data, so figures are planning estimates rather than turn-by-turn navigation.

4. What the System Does - Feature Overview

The dashboard has four tabs. All computation is performed by the server-side engine; the browser only collects input, draws the map and charts, and calculates simple estimates (time = distance ÷ 55 km/h, fuel cost = distance × ₹12/km).

Tab	Capabilities	Algorithms behind it
Route Planner	Generate a network of 5-40 real Indian cities · find the shortest route between any two cities · plan a multi-stop tour from a depot, choosing Quick / Better / Best · see the route drawn on an OpenStreetMap map · distance, drive-time and fuel-cost cards · tap cities or chips to add/remove stops	Haversine graph build, 3-nearest-neighbour links, Union-Find, Dijkstra, Nearest Neighbour, 2-Opt, Held-Karp
Orders & Drivers	Add / delete orders (from, to, kg, priority) · add / delete drivers (name, capacity) · load sample data · one-click assignment · per-order driver result · animated driver load bars	Priority ordering, greedy first-fit
Load Planner	Define a vehicle capacity and a list of packages (weight, value in ₹) · find the most valuable subset that fits · see chosen packages and fill percentage	0/1 Knapsack DP with traceback
Benchmarks	Enter any list of city counts (e.g. 5, 8, 12, 20, 30) · runs Nearest Neighbour, 2-Opt and Exact on each · bar chart of tour length, log-scale line chart of run time, detailed table with % improvement	All TSP planners, timing instrumentation

A coloured pill in the header shows which engine is active: **“C++ engine”** (binary found) or **“Python engine”** (fallback). Both expose the same commands, so the user experience is identical.

5. System Architecture

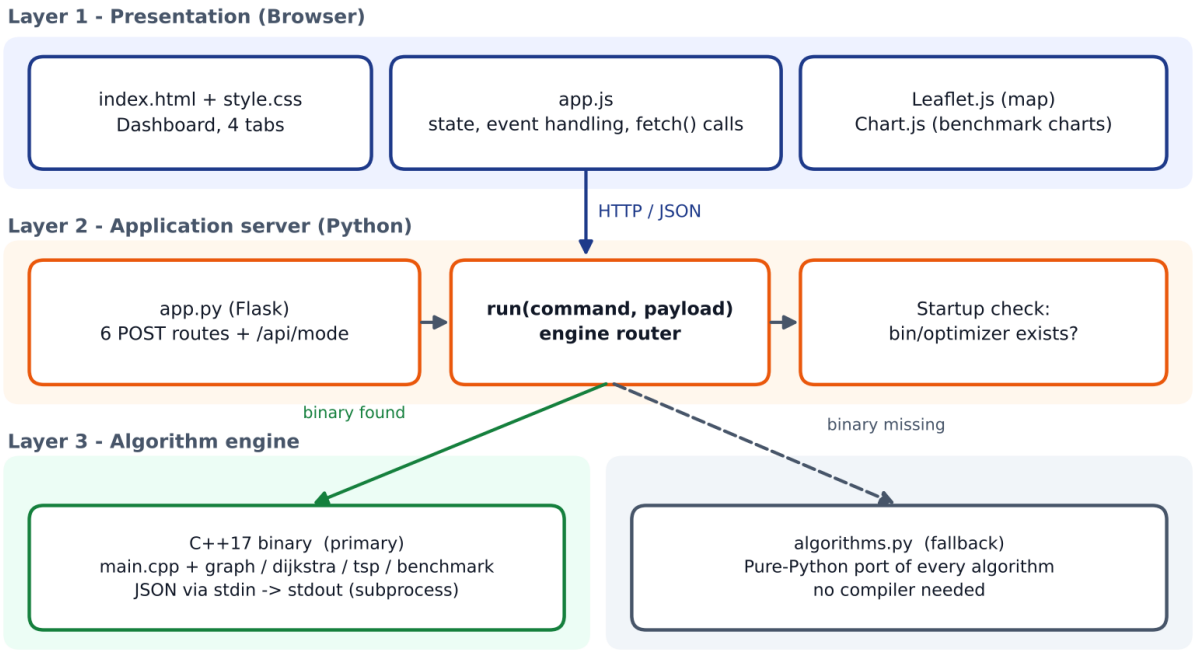


Figure 1 - Three-layer architecture with dual (C++ / Python) algorithm engine

5.1 Request life-cycle

- The browser sends a JSON POST to one of the Flask routes (e.g. `/api/tsp`).
- Flask builds a payload and calls `run(command, payload)`.
- **If** `bin/optimizer(.exe)` **exists** at start-up, the server launches it as a subprocess, writes the payload to *stdin* and reads one JSON document from *stdout* (30-second timeout; errors are returned as `{"error": ...}`).
- **Otherwise** the same command is handled in-process by `algorithms.run_command()`, a pure-Python port.
- The JSON result goes back to the browser, which draws the map layers, tables and charts.

5.2 Design decisions worth noting

- **Stateless engine.** Each call rebuilds the graph from the parameters (*n*), so no database or session is needed; orders, drivers and packages live in browser memory.
- **Graceful degradation.** The project runs with only `pip install flask`; the C++ build is an optional speed-up.
- **One command vocabulary** (`generate, shortest, tsp, assign, benchmark, knapsack`) shared by both engines keeps them behaviourally equivalent.
- **Front-end helper.** A small client-side Dijkstra (array version) is used only to draw each tour leg along real road links on the map.

6. Technology Stack

Layer	Technology	Used for
Algorithm engine (primary)	C++17, STL (`priority_queue, vector, sort`), `<chrono>`	Fast graph, routing, DP and benchmarking code
JSON in C++	nlohmann/json 3.11.3 (header-only, fetched by CMake)	Parsing stdin / building stdout JSON
Build system	CMake ≥ 3.14, MinGW-w64 g++ (Windows), `build.bat`	Produces `bin/optimizer(.exe)`
Algorithm engine (fallback)	Python ≥ 3.9 standard library (`heapq, math, random, time`)	Pure-Python port in `algorithms.py`
Web server	Flask ≥ 3.0.0 (only dependency in `requirements.txt`)	REST API, serving the dashboard, engine routing
Front end	HTML5, CSS3 (custom design, Inter font), vanilla JavaScript (ES6+)	Dashboard, state handling, fetch API calls
Maps	Leaflet 1.9.4 + OpenStreetMap tiles	Interactive map, route and tour overlays
Charts	Chart.js 4.4.0	Benchmark cost bars and run-time line chart
Tooling	Windows batch scripts (`build.bat, run.bat`), Git, VS Code settings	One-click build and launch

Internet dependency. Leaflet, Chart.js, Google Fonts and OpenStreetMap tiles are loaded from CDNs, so the dashboard needs an internet connection in the browser. The C++ build also needs internet once, to fetch nlohmann/json.

7. Project Structure

```
t3-daa-project/
├── app.py                    Flask backend, engine router           (113 lines)
├── algorithms.py             Pure-Python engine, all algorithms     (390 lines)
├── requirements.txt          flask>=3.0.0
├── build.bat / run.bat       Windows build and launch scripts
├── templates/
│   └── index.html            Dashboard markup, 4 tab panels         (221 lines)
├── static/
│   ├── css/style.css         Dashboard styling                      (307 lines)
│   └── js/app.js             Front-end logic, map, charts           (423 lines)
└── cpp/
    ├── CMakeLists.txt        Build configuration (C++17)
    ├── main.cpp              CLI entry: reads JSON, runs a command  (312 lines)
    ├── include/
    │   ├── order.h           Order struct, Priority and Status enums
    │   ├── driver.h          Driver struct with canTake()
    │   ├── graph.h           Edge, Graph (adjacency list, Dijkstra)
    │   └── optimizer.h       TSP + benchmark declarations
    └── src/
        ├── core/order_manager.cpp     OrderManager, priority queue   (118)
        ├── routing/graph.cpp          Graph + Dijkstra               (73)
        ├── routing/dijkstra.cpp       JSON wrappers for paths        (49)
        ├── routing/tsp.cpp            NN, 2-Opt, Held-Karp           (128)
        ├── analysis/benchmark.cpp     Benchmark + knapsack DP        (105)
        └── analysis/comparator.cpp    Multi-size comparison runner   (35)
```

7.1 File-to-role mapping

Area	Files / functions	Owner
Orders, drivers, priority, assignment	`order.h`, `driver.h`, `order_manager.cpp`; `main.cpp` → `assign`; `algorithms.assign_orders()`; Orders & Drivers tab in `app.js`	Member 1 - Piyush
Graph, Dijkstra, TSP heuristics	`graph.h`, `graph.cpp`, `dijkstra.cpp`, `tsp.cpp` (NN, 2-Opt); `algorithms.py`: graph builders, `dijkstra`, `tsp_nearest_neighbor`, `two_opt`; Route Planner tab	Member 2 - Varun
DP, benchmarks, comparison, visualization	`benchmark.cpp`, `comparator.cpp`, `optimizer.h`, `tspDP` (Held-Karp); `algorithms.py`: `tsp_dp`, `knapsack_dp`, `benchmark_single`; Load Planner and Benchmarks tabs, charts	Member 3 - Agamya
Shared integration	`app.py`, `index.html`, `style.css`, `main.cpp` dispatcher, build scripts	Team

8. How It Works - Module Walkthrough

8.1 Building the road network (graph model)

The graph is **undirected and weighted**. Nodes are the first *n* cities (2 ≤ n ≤ 40) from a built-in table of Indian cities with latitude and longitude (Delhi, Mumbai, Bengaluru, Chennai, Kolkata, Hyderabad, …). Edges are built in three steps:

- **Distance.** For every pair of cities the *haversine* great-circle distance is computed and multiplied by a **1.3 road factor** (roads are longer than straight lines), rounded to 0.1 km.
- **Sparse links.** Each city is joined to its **3 nearest** cities. This keeps the graph sparse so Dijkstra finds genuine multi-hop routes instead of one direct edge.
- **Guaranteed connectivity.** A **Union-Find (disjoint-set)** structure detects separate components; the shortest missing edge between two components is repeatedly added until the graph is connected.

Examples of resulting sizes: 5 cities → 8 roads, 12 cities → 23 roads, 20 cities → 35 roads, 40 cities → 74 roads. The graph is stored as an **adjacency list**.

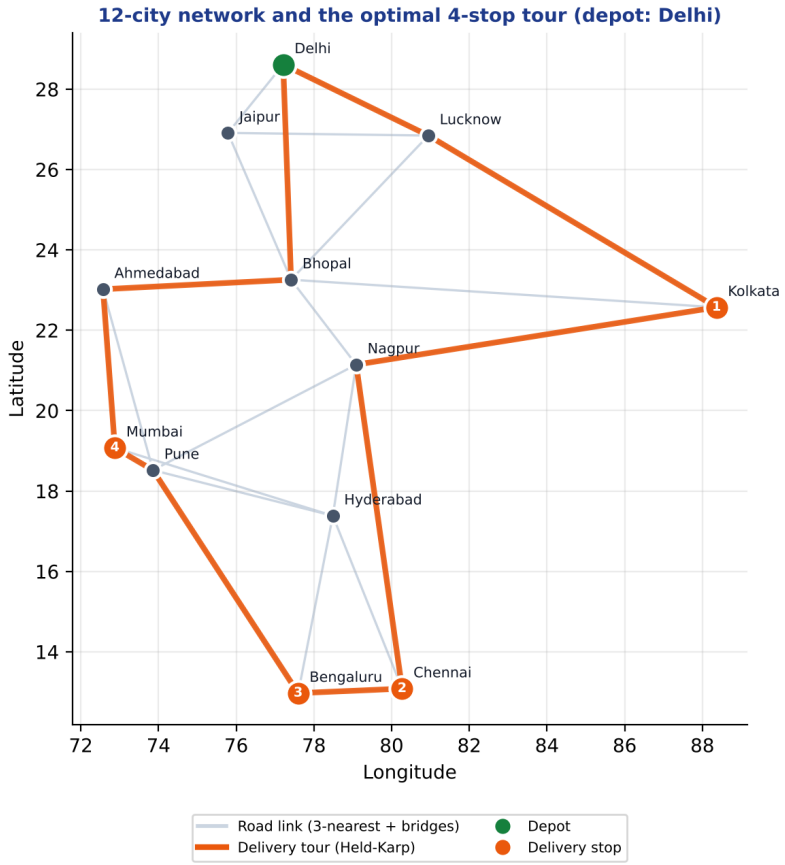


Figure 2 - The 12-city network (grey roads) and the optimal 4-stop tour from Delhi; legs follow real road links

8.2 Shortest path - Dijkstra's algorithm

Given a source city, Dijkstra's algorithm maintains a distance array and a **min-heap** of (distance, node). The smallest-distance node is popped, and each neighbour is relaxed if a shorter path is found; stale heap entries are skipped (`if d > dist[u]: continue`). A *prev* array lets the path be rebuilt by walking back from the destination.

```
dist[src] = 0;  heap.push((0, src))
while heap not empty:
    (d, u) = heap.pop_min()
    if d > dist[u]: continue            # stale entry
    for (v, w) in adj[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w;  prev[v] = u
            heap.push((dist[v], v))
path = walk prev[] back from dst        # reverse to get src -> dst
```

Complexity: O((V + E) log V). Dijkstra is also the *distance oracle* for every other routing algorithm in the project.

8.3 Multi-stop delivery - the TSP planners

The vehicle starts at a **depot**, must visit every selected stop once and return. The cost between two stops is the Dijkstra shortest distance between them, so tours may pass through intermediate cities that are not stops. Three planners are offered:

Planner (UI label)	Idea	Time	Quality
Quick - Nearest Neighbour	From the current city always go to the nearest unvisited stop; finally return to depot	O(m · (V+E) log V)	Fast, but typically 12-19% above optimal in this project's benchmarks
Better - 2-Opt	Start from the NN tour; repeatedly reverse a segment tour[i..j] if that shortens the two boundary edges; stop when no swap improves	O(m^2 · (V+E) log V) per pass (as coded)	Local optimum; never worse than NN
Best - Held-Karp DP (exact)	DP over subsets: dp[S][i] = cheapest way to start at depot, visit set S and end at i; bitmask encodes S; parent table rebuilds the tour	O(2^m · m^2) time, O(2^m · m) space	Provably optimal; limited to ≤ 12 stops

```
# Held-Karp (exact TSP)            m = number of stops, depot = node 0
dp[1<<i][i] = dist[depot][i]                    for each stop i
for S in 1 .. 2^m-1:                            # every subset
  for i in S:                                   # last visited stop
    for j not in S:                             # next stop
      dp[S | 1<<j][j] = min(dp[S | 1<<j][j], dp[S][i] + dist[i][j])
answer = min over i of dp[FULL][i] + dist[i][depot]  # then rebuild via parent[][]
```

> **Safety rule.** If the user picks "Best" with more than 12 stops, the UI automatically falls back to 2-Opt and shows a notice, because the DP table grows as 2^m.

8.4 Orders, priority and driver assignment

Each order has *source, destination, weight (kg)* and a **priority**: URGENT = 2, NORMAL = 1, LOW = 0. Each driver has a *name* and *capacity (kg)*. Assignment is a **greedy** procedure: orders are taken in descending priority and each is given to the **first driver** whose remaining capacity is at least the order's weight. If no driver qualifies the order is reported as **UNASSIGNED** ("No capacity"). The result lists every order with its driver, plus each driver's load against capacity.

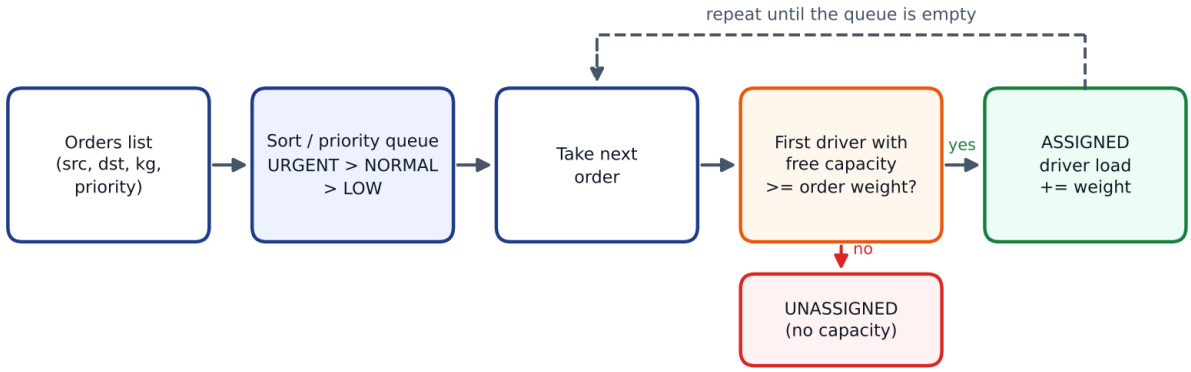


Figure 3 - Priority ordering followed by greedy first-fit driver assignment

Complexity: ordering O(n log n) + greedy matching O(n · d) for n orders and d drivers. In the C++ `OrderManager` class the ordering is done with a max-priority-queue (heap) with the comparator *higher priority first, then lower order id*; the live request path (`main.cpp` and `algorithms.py`) achieves the same ordering with a sort.

8.5 Load planning - 0/1 Knapsack (dynamic programming)

When not all parcels fit in one vehicle, the Load Planner picks the subset with **maximum total value** whose total weight does not exceed capacity *W*. A table dp[i][w] stores the best value using the first *i* packages with capacity *w*: `dp[i][w] = max(dp[i-1][w], dp[i-1][w - wt[i]] + val[i])`. A backward **traceback** through the table recovers which packages were chosen. Complexity O(n · W) time and space. Unlike a greedy “heaviest value first” rule, the DP is guaranteed optimal.

8.6 Benchmarking and comparison

For each requested size *n* the benchmark builds a random complete Euclidean graph of *n* cities (fixed seed 42, reproducible), takes up to 10 stops (cities 1...min(n-1, 10)), and times the three planners on the identical instance. It reports tour cost, run time in milliseconds, and the **percentage saved versus Nearest Neighbour**. The Exact planner is included only when the stop count is small enough. Results feed the two charts and the detailed table.

9. Data Structures & Algorithms Used

9.1 Algorithms catalogue

Algorithm / technique	Paradigm	Time complexity	Where used	Owner
Dijkstra's shortest path	Greedy + heap	O((V+E) log V)	Shortest-route tab; distance oracle for all TSP planners	2
Nearest-Neighbour TSP	Greedy heuristic	O(m · Dijkstra)	“Quick” planner; starting tour for 2-Opt	2
2-Opt local search	Local search / improvement	O(m²) swaps per pass × distance look-ups	“Better” planner	2
Greedy driver assignment (first-fit)	Greedy	O(n · d)	Orders & Drivers tab	1
Priority ordering (URGENT › NORMAL › LOW)	Heap / sorting	O(n log n)	Before assignment; C++ priority queue in OrderManager	1
Held-Karp exact TSP	Dynamic programming + bitmask	$O(2^m \cdot m^2)$, space $O(2^m \cdot m)$	“Best” planner; benchmark baseline	3
0/1 Knapsack	Dynamic programming	O(n · W)	Load Planner tab	3
Benchmarking & comparison	Empirical analysis	-	Benchmarks tab (timers: chrono / perf_counter)	3
k-Nearest-Neighbour graph construction	Sorting + greedy	O(n² log n)	Road-link generation (k = 3)	2
Union-Find (disjoint-set, path halving)	Disjoint-set forest	≈ O(α(n)) per op	Connect separate graph components	2
Haversine formula	Geometry	O(1) per pair	Distance between city coordinates	2

9.2 Data structures catalogue

Data structure	Where it appears	Purpose
Adjacency list	`Graph::adj` (vector of vector<Edge>); Python `adj`; JS `adjList`	Memory-efficient sparse road network; O(deg) neighbour scan
Min-heap	`std::priority_queue<…, greater<>>` in `graph.cpp`; Python heapq	Extract the nearest unsettled node in Dijkstra
Max-heap / priority queue	`OrderCmp` + `std::priority_queue<Order*>` in `order_manager.cpp`	Always surface the most urgent pending order
Hash maps / sets	Python `dict` (driver loads), JS object `weightOf` (edge weights), JS `Set` (selected stops)	O(1) lookup of loads, edge weights and stop membership
2-D DP tables	Knapsack `dp[n+1][W+1]`; Held-Karp `dp[2^m][m]` and `parent[2^m][m]`	Store sub-problem results; parent table enables path reconstruction
Bitmask	Held-Karp subset S (`S & (1<<i)`)	Compact subset representation (2^m states)
Result / distance matrices	Pairwise `dist[N][N]` in Held-Karp; benchmark result rows	Pre-computed shortest distances; structured comparison output
Predecessor array	`prev[]` in Dijkstra	Rebuild the actual path, not just its length
Disjoint-set (parent array)	`comp[]` with `find()` in the graph builders	Component tracking for connectivity
Structs & enums	`Order, Driver, Edge, Priority, Status`	Typed domain model for the Foundation layer

Note on linked lists. The team plan lists linked lists for Member 1. The current code stores orders and drivers in contiguous vectors / arrays (a better fit for random access and sorting), so no linked list is present in the implementation.

10. Team Roles & Responsibilities

The work was divided into three layers that build on each other: Member 1 provides the **data and order model**, Member 2 the **graph and routing engine**, and Member 3 the **optimization, comparison and visualization layer**. Each subsection below first states the responsibilities as planned by the team, then maps them to what is actually present in the code base.

Member 1 - Piyush Mahapatra — Core Data & Order Management · "The Foundation Layer"

Planned responsibilities

- **Order management system** - create, update, cancel orders; assign priority levels (urgent / normal / low)
- **Priority queue** - min/max heap for order scheduling; ensure urgent packages bubble up
- **Driver / agent management** - availability, capacity, assignment logic (greedy matching)
- **Delivery status tracker** - order life-cycle: pending → assigned → in-transit → delivered

Data structures: heaps, hash maps, linked lists. **Algorithms:** greedy driver assignment, sorting by priority + deadline.

Delivered in the code base

Responsibility	What exists in the project	Status
Create orders, priority levels	UI form (from, to, kg, priority) · `Order` struct with `Priority {LOW, NORMAL, URGENT}` · `OrderManager::addOrder()`	**Implemented**
Cancel orders	Delete (×) button removes an order from the list before assignment	**Implemented**
Update / edit orders	No edit form; an order is changed by deleting and re-adding it	**Not implemented**
Priority queue (heap)	`OrderCmp` comparator with `std::priority_queue` inside `OrderManager::assignOrders()`; the live path (`main.cpp`, `algorithms.py`) uses an equivalent priority sort	**Partial**
Driver management	`Driver` struct (capacity, current load, availability, assigned orders, `canTake()`) · UI to add / remove drivers · load bars	**Implemented**
Greedy driver assignment	First-fit by priority; unassigned orders reported; per-driver load summary	**Implemented**
Delivery status tracker	`Status {PENDING, ASSIGNED, IN_TRANSIT, DELIVERED}` defined; PENDING → ASSIGNED transition coded in `OrderManager`; later transitions and a tracker view are not built	**Partial**
Deadline-aware sorting	Ordering uses priority (ties by id / entry order); no deadline field exists yet	**Not implemented**

Member 2 - Varun Verma

Graph & Routing Engine · “The Brain”

Planned responsibilities

- **City / map graph** - weighted graph from delivery locations (nodes = locations, edges = roads with distance/time weights)
- **Dijkstra's algorithm** - single-source shortest path from warehouse/hub to delivery points
- **Multi-stop route planning** - given N stops for one driver, find an efficient route
- **TSP heuristics** - nearest neighbour and 2-opt swap for small instances (≤ 20 nodes)

Data structures: adjacency list, min-heap (for Dijkstra). **Algorithms:** Dijkstra, TSP nearest-neighbour, 2-opt.

Delivered in the code base

Responsibility	What exists in the project	Status
City / map graph	Weighted undirected graph of up to 40 real Indian cities (lat/lng); edges = road links weighted in km (haversine × 1.3); 3-nearest links + Union-Find bridging; adjacency list	**Implemented**
Time weights	Edges store distance only; drive time is derived in the UI at 55 km/h and fuel cost at ₹12/km	**Partial**
Dijkstra	Min-heap implementation in C++ (`Graph::dijkstra`, `dijkstraPath`) and Python (`dijkstra`, `reconstruct_path`); usable between *any* two cities, with path drawn on the map	**Implemented**
Multi-stop route planning	Tour from a depot through any chosen stops and back; legs drawn along real road links; km / time / cost summary	**Implemented**
Nearest-neighbour TSP	`tspNearestNeighbor` / `tsp_nearest_neighbor`	**Implemented**
2-Opt improvement	`twoOpt` / `two_opt` - repeated segment reversal until no improvement	**Implemented**
“≤ 20 nodes” bound	No hard cap in the heuristics; they accept any number of stops (UI limit: 40 cities). 2-Opt becomes slow as stops grow because it re-runs Dijkstra for every candidate swap	**Partial**

Member 3 - Agamya Jain

Optimization, Comparison & Visualization · “The Analyst”

Planned responsibilities

- **Dynamic programming** - DP for small capacity / knapsack-style sub-problems (e.g. fitting packages into vans)
- **Scalability benchmarks** - run the routing engine on 5, 10, 50, 100 locations; measure time and cost
- **Route comparison module** - compare Dijkstra vs greedy vs TSP heuristic on the same input; output cost / distance / time
- **CLI / simple UI** - display routes, maps (ASCII or matplotlib), summary stats

Data structures: DP tables, result matrices. **Algorithms:** DP, brute-force (for ≤ 10 nodes as baseline), benchmarking.

Delivered in the code base

Responsibility	What exists in the project	Status
0/1 Knapsack DP	`knapsackDP` / `knapsack_dp` with traceback; Load Planner tab with sample packages and fill-percentage bar	**Implemented**
Exact TSP by DP	Held-Karp bitmask DP (`tspDP` / `tsp_dp`), up to 12 stops - serves as the optimal baseline	**Beyond brief**
Scalability benchmarks	Runs any list of sizes (default 5, 8, 12, 20, 30; sizes such as 5, 10, 20, 50 and 100 also work); measures cost and milliseconds; stops capped at 10 per run	**Implemented**
Route comparison module	Nearest Neighbour vs 2-Opt vs Exact on the same instance, with cost, time and % improvement; Dijkstra is the shared distance engine rather than a separate contender	**Partial**
UI / maps / stats	Full web dashboard instead of CLI: Leaflet map, route and tour overlays, Chart.js charts, summary cards (distance, drive time, fuel cost)	**Beyond brief**
Brute-force baseline (≤ 10 nodes)	Not coded; the exact Held-Karp DP plays the baseline role instead	**Not implemented**
DP tables, result matrices	Knapsack and Held-Karp tables; pairwise distance matrix; benchmark result rows	**Implemented**

Reading the status column. **Implemented** = present and working in the delivered code; **Partial** = core idea present, some aspects of the plan missing; **Not implemented** = in the plan but not in the code; **Beyond brief** = delivered in addition to the plan.

11. User Flow

The interface is a single page. On opening, the app automatically draws a 12-city network and shows a default Delhi → Chennai shortest route, so the user sees a working result immediately. From there the user chooses one of four tabs; each tab is a short, linear workflow.

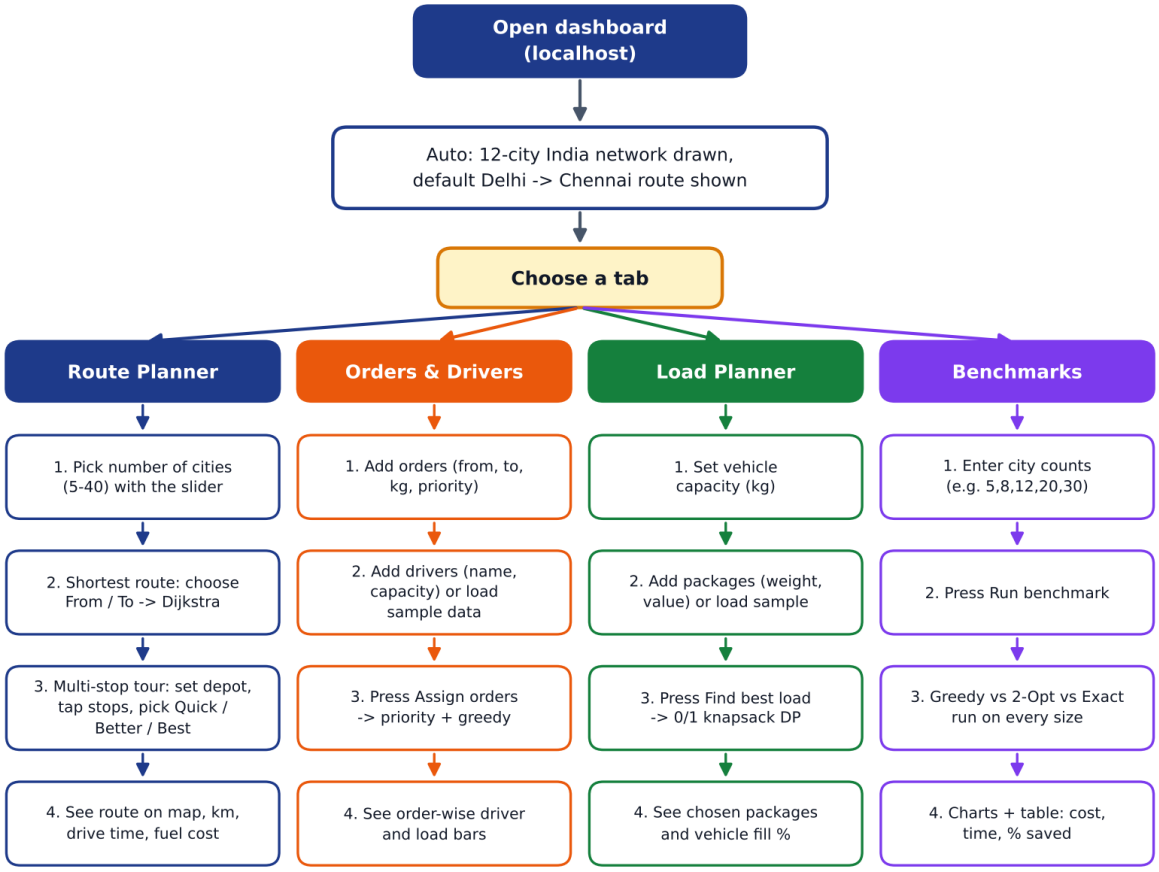


Figure 4 - End-to-end user flow across the four tabs

11.1 Step-by-step by tab

A. Route Planner

- **1 · Cities:** drag the slider (5-40) - a new network is generated and drawn.
- **2 · Shortest route:** choose *From* and *To* (a swap button helps) and press *Find shortest route*. The blue path appears with distance, drive time, fuel cost and a city-by-city timeline.
- **3 · Multi-stop delivery:** choose the depot, tap city chips (or map markers) to add / remove stops, pick *Quick*, *Better* or *Best*, and press *Plan delivery tour*. The orange tour is drawn with numbered stops and a leg-by-leg timeline.

B. Orders & Drivers

- Add orders with *From, To, Kg, Priority*; add drivers with *Name, Capacity* (or press *Load sample data*: 7 orders and 4 drivers).
- Press *Assign orders*. The result table shows each order with its priority badge and driver (or “No capacity”), and each driver gets an animated load bar (turns amber at ≥ 80% and red at 100%).

C. Load Planner

- Set the vehicle capacity, add packages with weight and value (or load sample packages), press *Find best load*.

- Chosen packages are highlighted with ✓, others greyed; the badge shows the best total value and a bar shows how full the vehicle is.

D. Benchmarks

- Type city counts separated by commas and press *Run benchmark*; a spinner is shown while the server computes.
- Two charts (tour length, run-time on log scale) and a table with '2-Opt saves' and 'Exact saves' percentages are drawn.

11.2 Example session - a dispatcher's morning

- Open the dashboard and, in **Orders & Drivers**, enter the day's orders; mark time-critical ones *Urgent*. Add the available drivers with their van capacities.
- Press *Assign orders* - urgent parcels are placed first and any parcel that does not fit anyone is flagged for a second vehicle.
- Go to the **Load Planner** for an over-subscribed van to choose the most valuable parcels that fit.
- Go to **Route Planner**, set the hub as depot, tap the delivery cities and run *Best* (≤ 12 stops) or *Better* to get the shortest tour with distance, time and fuel estimate.
- Optionally open **Benchmarks** to justify the algorithm choice to management or in a viva.

12. REST API Reference

All endpoints accept and return JSON. `GET /` serves the dashboard and `GET /api/mode` returns `{"mode": "cpp" | "python"}`.

Endpoint (POST)	Request body	Response (summary)
`/api/generate`	`{n, seed}` - n clamped to 2-40	`{nodes[], edges[], n}` - city names, lat/lng, road weights
`/api/shortest`	`{n, seed, src, dst}`	`{cost, reachable, src_name, dst_name, path[]}`
`/api/tsp`	`{n, seed, depot, stops[], algo}` with algo = greedy | 2opt | dp	`{algo, cost, tour[]}` - tour starts and ends at depot
`/api/assign`	`{orders[{src,dst,weight,priority}], drivers[{name,capacity}]}`	`{assignments[], drivers[{id,name,capacity,load}]}`
`/api/benchmark`	`{sizes[]}` (default 5, 10, 20, 50)	Array of `{n_cities, n_stops, *_cost, *_ms, improvement_*_pct}`
`/api/knapsack`	`{weights[], values[], capacity}`	`{max_value, selected[], total_weight}`

13. Verified Sample Results

To make this document factual, the algorithms were executed on the delivered code (Python engine) with the same sample data that the dashboard's "Load sample" buttons use. Distances are in km on the 12-city network; run-times vary slightly from machine to machine.

13.1 Shortest path

Delhi → Chennai = 2,330.3 km via **Delhi → Bhopal → Nagpur → Chennai** (the default route shown when the dashboard opens). At 55 km/h this is about 42 h 22 m of driving and roughly ₹27,964 in running cost at ₹12/km.

13.2 Multi-stop tour: heuristic vs exact

Instance (depot Delhi)	Nearest Neighbour	2-Opt	Exact (Held-Karp)	Observation
4 stops: Kolkata, Mumbai, Bengaluru, Chennai (idx 1-4)	7,738.3 km	7,613.3 km	7,613.3 km	2-Opt reaches the optimum; saves 1.6%
8 stops (idx 1-8)	8,065.7 km	8,065.7 km	7,756.7 km	2-Opt stuck in a local optimum; exact is 3.8% better

The second row is the most instructive: it shows *why* the project offers an exact planner - a local-search heuristic can stop at a tour that is not globally optimal.

13.3 Greedy assignment on the sample data

Order	Priority	Weight	Assigned driver
#1	Urgent	30 kg	Arjun (120 kg)
#2	Urgent	80 kg	Arjun (120 kg)
#3	Urgent	35 kg	Priya (100 kg)
#4	Normal	50 kg	Priya (100 kg)
#5	Normal	15 kg	Priya (100 kg)
#6	Low	20 kg	Rahul (80 kg)
#7	Low	45 kg	Rahul (80 kg)

All 7 orders are assigned. Final loads: Arjun 110/120 kg, Priya 100/100 kg, Rahul 65/80 kg, Neha 0/60 kg - the first-fit rule fills earlier drivers first and leaves the last driver unused. (Order numbers here are positions after priority sorting.)

13.4 Knapsack load planning

Capacity 50 kg with 7 sample packages: the DP selects **Electronics (10 kg, ₹8,000), Medicines (20 kg, ₹10,000), Textiles (5 kg, ₹4,000) and Fresh produce (15 kg, ₹9,000)** - exactly **50 kg** for a maximum value of **₹31,000**.

13.5 Benchmark

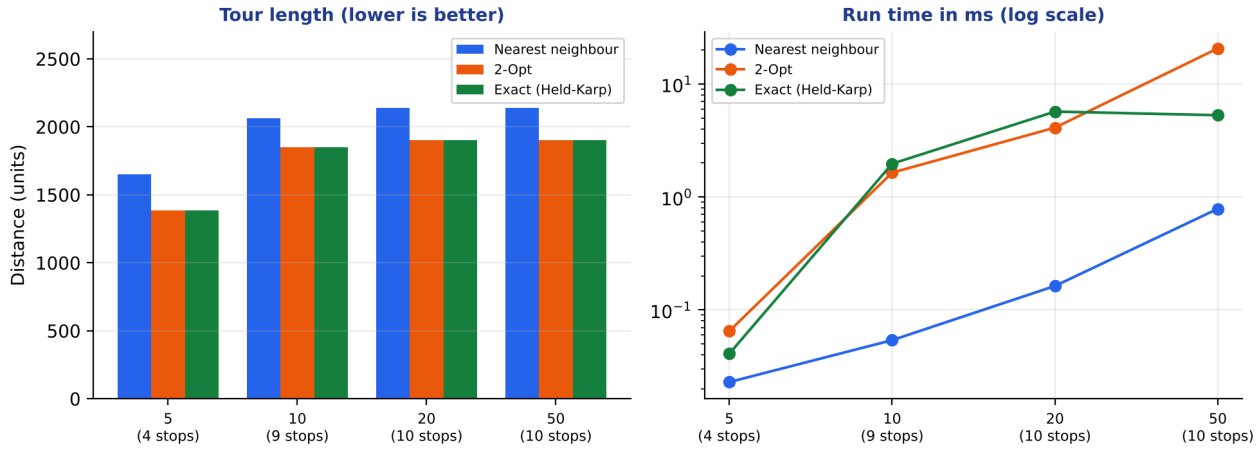


Figure 5 - Tour length and run time of the three planners (random Euclidean networks, ≤ 10 stops)

Cities	Stops	NN cost	2-Opt cost	Exact cost	2-Opt saves	NN ms	2-Opt ms	Exact ms
5	4	1652.7	1384.1	1384.1	16.2%	0.02	0.07	0.04
10	9	2062.3	1848.6	1848.6	10.4%	0.05	1.64	1.97
20	10	2137.4	1904.7	1904.7	10.9%	0.16	4.11	5.70
50	10	2137.4	1904.7	1904.7	10.9%	0.79	20.66	5.29

- **Quality:** 2-Opt and Exact give identical tours on these instances and both beat Nearest Neighbour by about 10-16%.
- **Speed:** Nearest Neighbour is the fastest by one to two orders of magnitude; Exact cost grows with the number of stops (2^m), not with the number of cities.
- **Why the 20- and 50-city rows have the same costs:** the benchmark caps the stops at 10, so both rows route the same first 10 cities; the larger graph mainly increases the Dijkstra work (visible in the 2-Opt time, 4.1 ms → 20.7 ms).

> A 100-city run was also verified: 2-Opt ≈ 81 ms, Exact ≈ 10 ms, Nearest Neighbour ≈ 3 ms (≤ 10 stops).

14. Installation & Running

14.1 Quick start - Python only (no compiler)

```
pip install flask
python app.py          # starts the server and uses the Python engine
```

Open the address printed by Flask in a browser. The header pill will read **Python engine**.

14.2 Full setup - C++ engine (Windows)

- Install **CMake ≥ 3.14**, **MinGW-w64 (g++)** and **Python ≥ 3.9**; add them to PATH.
- Run `build.bat` - configures CMake (downloads nlohmann/json), compiles, and writes `bin\optimizer.exe`.
- Run `run.bat` - checks Python and Flask, detects the binary, opens the browser and starts `app.py`.
- The header pill now reads **C++ engine**.

> **Port note.** `app.py` starts Flask on **port 5001**, while `README.md` and `run.bat` refer to port 5000. If the page does not open, browse to `http://localhost:5001` (or change one of the two to match).

15. Known Limitations & Future Scope

15.1 Current limitations

- **No persistence.** Orders, drivers and packages live in browser memory and disappear on refresh; there is no database or login.
- **Assignment ignores geography and deadlines.** Greedy first-fit looks only at priority and weight; it does not consider distance between orders and drivers, time windows or deadlines.
- **Status life-cycle is partial.** Only PENDING → ASSIGNED is modelled; in-transit and delivered updates, and order editing, are not built.
- `OrderManager` **not linked.** `order_manager.cpp` is not in the CMake source list and is not included by `main.cpp`; the live assignment is a separate implementation in `main.cpp`. In the C++ path, ties inside a priority level are ordered by `std::sort` (not stable), whereas Python keeps entry order.
- **2-Opt efficiency.** Each candidate swap triggers three or four Dijkstra runs. Pre-computing an all-pairs distance matrix once would reduce the per-pass cost to $O(m^2)$.
- **Exact planner limit.** Held-Karp is exponential; it is capped at 12 stops (and the benchmark runs it only for small stop counts).
- **Benchmark realism.** The benchmark uses random Euclidean complete graphs, not the Indian road network, and caps stops at 10.
- **Estimates only.** Distances are great-circle × 1.3; the seed parameter does not change the Indian-city network; time and fuel use fixed constants (55 km/h, ₹12/km).
- **Online assets.** Map tiles, Leaflet, Chart.js and fonts need internet access.

15.2 Future enhancements

- Wire `OrderManager` into the build; add order editing, full status tracking (in-transit, delivered) and deadlines in the priority comparator.
- Multi-vehicle routing (VRP): split stops among drivers, then optimise each driver's tour.

- Add a brute-force baseline for ≤ 10 nodes, and stronger heuristics (Or-opt, 3-opt, Christofides) for comparison.
- A* search with a haversine heuristic; time-dependent edge weights; integration with a real road-routing service.
- Persistence (SQLite / PostgreSQL), user accounts, CSV import/export of orders, and PDF route sheets for drivers.
- Automated unit tests that run the C++ and Python engines on identical inputs to guarantee equivalence.

16. Conclusion

DeliverOpt India turns a collection of classical DAA topics - heaps, greedy algorithms, graph shortest paths, local search, bitmask and table-based dynamic programming - into a coherent, usable logistics application. The layered split of work (Foundation, Brain, Analyst) maps cleanly onto the code: Member 1's order and assignment layer feeds Member 2's graph and routing engine, whose planners are then measured, compared and visualised by Member 3. The dual C++/Python engine, the stateless JSON interface and the live benchmarks make the project easy to run, easy to explain and easy to extend toward a production-grade dispatch system.

Appendix - Glossary

Term	Meaning
Depot / hub	The start and end point of a delivery tour
Greedy algorithm	Makes the locally best choice at every step without revisiting it
Heap / priority queue	Structure that always returns the smallest (or largest) element quickly in O(log n)
TSP	Travelling Salesman Problem - shortest closed tour visiting each stop exactly once (NP-hard)
Heuristic	A fast method that gives a good, but not guaranteed optimal, answer
2-Opt	Improves a tour by reversing a segment whenever that reduces total length
Held-Karp	Exact dynamic-programming algorithm for TSP using subsets, $O(2^m \cdot m^2)$
Knapsack	Choose items with maximum value subject to a weight limit
Haversine	Formula for the great-circle distance between two latitude/longitude points
Union-Find	Disjoint-set structure that tracks which nodes are already connected